

Portable Executable

Portable Executable (PE) is a file format for native executable code on 32-bit and 64-bit Windows operating systems, as well as in UEFI environments.^[2] It is used for native executables (.exe, .com), dynamic link libraries (.dll, .ocx), system drivers (.sys, .drv) and many other types of files. The PE format supports storing the data required to load and start an operating system process – including references to dynamic link libraries, tables for importing and exporting application programming interface (API) functions, resource management data and thread-local storage (TLS) information.

Portable Executable

<u>Filename extension</u>	<u>.exe</u> , <u>.com</u> , <u>.dll</u> , <u>.sys</u> , <u>.acm</u> , <u>.ax</u> , <u>.cpl</u> , <u>.drv</u> , <u>.efi</u> , <u>.ime</u> , <u>.mui</u> , <u>.ocx</u> , <u>.scr</u> , <u>.tsp</u> , <u>.msstyles</u>
<u>Internet media type</u>	application/vnd.microsoft.portable-executable ^[1]
<u>Magic number</u>	4D 5A (MZ in <u>ASCII</u>) and 50 45 00 00 (PE)
<u>Developed by</u>	Currently: <u>Microsoft</u>
<u>Type of format</u>	Binary, executable, <u>object</u> , <u>shared libraries</u>
<u>Extended from</u>	<u>DOS MZ executable</u> <u>COFF</u>

According to the Unified Extensible Firmware Interface (UEFI) specification, the PE format is also the accepted standard for executables in EFI environments.^[3] On Windows NT systems, it currently supports a range of instruction sets, including IA-32, x86-64 (AMD64/Intel 64), IA-64, ARM and ARM64. Before the advent of Windows 2000, Windows NT (and by extension the PE format) also supported MIPS, Alpha, and PowerPC architectures. Moreover, thanks to its use in Windows CE, PE has maintained compatibility with several MIPS, ARM (including Thumb), and SuperH variants.^[4]

Functionally, the PE format is similar to other platform-specific executable formats, such as the ELF format used in Linux and most Unix-like systems, and the Mach-O format found in macOS and iOS.

History

Microsoft first introduced the PE format with Windows NT 3.1, replacing the older 16-bit New Executable (NE) format. Soon after, Windows 95, 98, ME, and the Win32s extension for Windows 3.1x, all adopted the PE structure. Each PE file includes a DOS executable header, which generally displays the message "This program cannot be run in DOS mode". However, this DOS section can be replaced by a fully functional DOS program, as demonstrated in the Windows 98 SE installer. Developers can add such a program using the /STUB switch with Microsoft's linker, effectively creating a fat binary.^[5]

Over time, the PE format has grown with the Windows platform. Notable extensions include the .NET PE format for managed code, PE32+ for 64-bit address space support, and a specialized version for Windows CE.

To determine whether a PE file is intended for 32-bit or 64-bit architectures, one can examine the Machine field in the IMAGE_FILE_HEADER.^[6] Common machine values are 0x014c for 32-bit Intel processors and 0x8664 for x64 processors. Additionally, the Magic field in the IMAGE_OPTIONAL_HEADER reveals whether addresses are 32-bit or 64-bit. A value of 0x10B indicates a 32-bit (PE32) file, while 0x20B indicates a 64-bit (PE32+) file.^[7]

Technical details

Layout

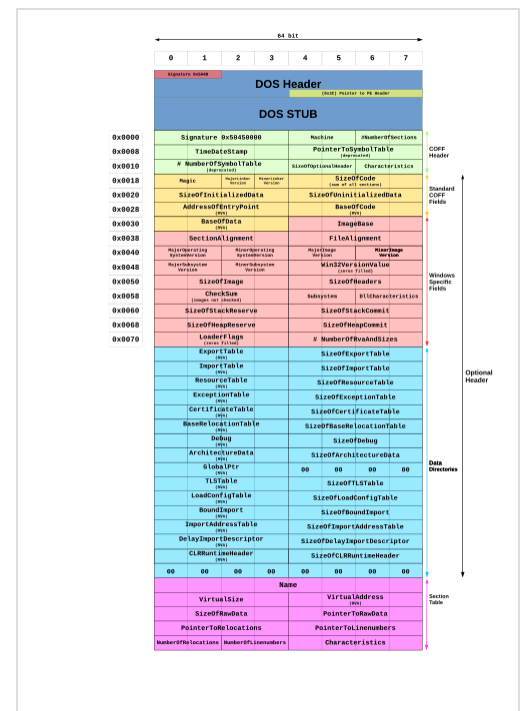
A PE file consists of several headers and sections that instruct the dynamic linker on how to map the file into memory. An executable image consists of several different regions, each requiring different memory protection attributes. To ensure proper alignment, the start of each section must align to a page boundary.^[8] For instance, the `.text` section, which contains program code, is typically mapped as an execute/read-only. Conversely, the `.data` section, which holds global variables, is mapped as no-execute/read write. However, to conserve space, sections are not aligned on disk in this manner. The dynamic linker maps each section to memory individually and assigns the correct permissions based on the information in the headers.^[9]

Import table

The *import address table* (IAT) is used as a lookup table when the application calls a function in a different module. The imports can be specified by ordinal or by name. Because a compiled program cannot know the memory locations of its dependent libraries beforehand, an indirect jump is necessary for API calls. As the dynamic linker holds modules and resolves dependencies, it populates the IAT slots with actual addresses of the corresponding library functions. Although this adds an extra jump, incurring a performance penalty compared to intermodular calls, it minimizes the number of memory pages that require copy-on-write changes, thus conserving memory and disk I/O. If a call is known to be intermodular beforehand (if indicated by a `dllimport` attribute), the compiler can generate optimized code with a simple indirect call opcode.^[9]

Address Space Layout Randomization (ASLR)

Modern operating systems use address space layout randomization (ASLR), a process that makes a PE file's in-memory layout unpredictable and therefore harder to exploit. During ASLR, the loader randomizes the virtual addresses where key components reside. This includes the executable's base, shared libraries, the heap, and the stack. Most PE files are not position-independent because mainstream compilers emit some absolute references relative to an assumed base. To cope with randomized rebasing, the linker stores a `.reloc` table that lets the loader adjust those references at load time.



Structure of a Portable Executable 32 bit

.NET, metadata, and the PE format

In a .NET executable, the PE code section contains a stub that invokes the [CLR](#) virtual machine startup entry, `_CorExeMain` or `_CorDllMain` in `mscorlib.dll`, much like it was in [Visual Basic](#) executables. The virtual machine then makes use of .NET metadata present, the root of which, `IMAGE_COR20_HEADER` (also called "CLR header") is pointed to by `IMAGE_DIRECTORY_ENTRY_COMHEADER` (the entry was previously used for [COM+](#) metadata in COM+ applications, hence the name) entry in the PE header's data directory. `IMAGE_COR20_HEADER` strongly resembles PE's optional header, essentially playing its role for the CLR loader.^[4]

The CLR-related data, including the root structure itself, is typically contained in the common code section, `.text`. It is composed of a few directories: metadata, embedded resources, strong names and a few for native-code interoperability. Metadata directory is a set of tables that list all the distinct .NET entities in the assembly, including types, methods, fields, constants, events, as well as references between them and to other assemblies.

Use on other operating systems

The PE format is also used by [ReactOS](#), an open-source operating system created to be binary-compatible with Windows. Historically, it has also been used by other operating systems such as [SkyOS](#) and [BeOS R3](#). However, both SkyOS and BeOS eventually moved to [ELF](#).

The [Mono development platform](#), which aims to be binary compatible with the Microsoft [.NET Framework](#), uses the same PE format as the Microsoft implementation. The same goes for Microsoft's own cross-platform [.NET Core](#).

On [x86\(-64\)](#) [Unix-like](#) operating systems, Windows binaries (in PE format) can be executed using [Wine](#). The [HX DOS Extender](#) also uses the PE format for native DOS 32-bit binaries, and can execute some Windows binaries in DOS, thus acting like an equivalent of Wine for DOS.

[Mac OS X 10.5](#) has the ability to load and parse PE files, although it does not maintain binary compatibility with Windows.^[10]

[UEFI](#) and UEFI firmware use PE files, as well as the platform related [ABI](#) and [calling convention](#) (x64 ABI for typical PC devices) for [applications](#).

See also

- [a.out](#) – Old Unix executable file format
- [Application virtualization](#) – Method to encapsulate computer programs
- [Comparison of executable file formats](#)
- [Executable compression](#) – Means of compressing an executable file

References

1. Andersson, Henrik (2015-04-23). "application/vnd.microsoft.portable-executable" (<https://www.iana.org/assignments/media-types/application/vnd.microsoft.portable-executable>). IANA. Retrieved 2017-03-26.
2. "Portable executable (PE) - Definition - Trend Micro IN" (<https://www.trendmicro.com/vinfo/in/security/definition/portable-executable-pe>). *www.trendmicro.com*. Retrieved 2022-11-10.
3. "UEFI Specification, version 2.8B" (<https://uefi.org/sites/default/files/resources/UEFI%20Spec%202.8B%20May%202020.pdf>) (PDF)., a note on p.15, states that "this image type is chosen to enable UEFI images to contain Thumb and Thumb2 instructions while defining the EFI interfaces themselves to be in ARM mode."
4. "PE Format (Windows)" ([https://msdn.microsoft.com/en-us/library/windows/desktop/ms680547\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms680547(v=vs.85).aspx)). Retrieved 2017-10-21.
5. "/STUB (MS-DOS Stub File Name)" (<http://msdn.microsoft.com/en-us/library/7z0585h5.aspx>). 3 August 2021.
6. PE trick explained: Telling 32 and 64 bit apart with naked eye (<https://gdatasoftware.com/blog/pebitnesstrick>) by Karsten Hahn
7. PE Format (<https://docs.microsoft.com/en-us/windows/win32/debug/pe-format>) at Microsoft.com
8. "The Portable Executable File From Top to Bottom" (<https://web.archive.org/web/20171020180919/http://www.csn.ul.ie/~caolan/pub/winresdump/winresdump/doc/pefile2.html>). Archived from the original (<http://www.csn.ul.ie/%7Ecaolan/pub/winresdump/winresdump/doc/pefile2.html>) on 2017-10-20. Retrieved 2017-10-21.
9. "Peering Inside the PE: A Tour of the Win32 Portable Executable File" (<https://msdn.microsoft.com/en-us/library/ms809762.aspx>). 30 June 2010. Retrieved 2017-10-21.
10. Chartier, David (2007-11-30). "Uncovered: Evidence that Mac OS X could run Windows apps soon" (<https://arstechnica.com/journals/apple.ars/2007/11/30/uncovered-evidence-that-mac-os-x-could-run-windows-apps-soon>). *Ars Technica*. Retrieved 2007-12-03. "... Steven Edwards describes the discovery that Leopard apparently contains an undocumented loader for Portable Executables, a type of file used in 32-bit and 64-bit versions of Windows. More poking around revealed that Leopard's own loader tries to find Windows DLL files when attempting to load a Windows binary."

External links

- PE Format (<https://docs.microsoft.com/en-us/windows/desktop/Debug/pe-format>) (latest online document, changes in time)
- Microsoft Portable Executable and Common Object File Format Specification. Revision 11.0, Jan 2017 (<https://download.microsoft.com/download/9/c/5/9c5b2167-8017-4bae-9fde-d599bac8184a/pecoff.docx>); Revision 10.0, Jun 2016 (https://bytepointer.com/download.php?name=pecoff_v10.docx); Revision 8.3, Feb 2013 (https://download.microsoft.com/download/9/c/5/9c5b2167-8017-4bae-9fde-d599bac8184a/pecoff_v83.docx); Revision 8.2, Sep 2010 (https://web.archive.org/web/20100923032233/http://download.microsoft.com/download/9/c/5/9c5b2167-8017-4bae-9fde-d599bac8184a/pecoff_v8.docx); Revision 8.1, Feb 2008 (https://web.archive.org/web/20081211013825/http://download.microsoft.com/download/9/c/5/9c5b2167-8017-4bae-9fde-d599bac8184a/pecoff_v8.docx); Revision 8.0, May 2006 (https://web.archive.org/web/20060706172855/http://download.microsoft.com/download/9/c/5/9c5b2167-8017-4bae-9fde-d599bac8184a/pecoff_v8.doc); Revision 6.0, Feb 1999 (<https://web.archive.org/web/20030724131335/http://www.microsoft.com/whdc/hwddev/download/hardware/pecoff.doc>); Revision 5.0, Oct 1997 (<https://bytepointer.com/resources/pec>

off_v5.0.htm); Revision 4.1, Aug 1994 (https://bytepointer.com/resources/pecoff_v4.1.htm); Revision 4.0, Sep 1993 (https://bytepointer.com/resources/pecoff_v4.0.htm)

- Tool Interface Standard (TIS) Formats Specifications for Windows Version 1.0 (https://openwatcom.org/ftp/devel/docs/pe_and_symbols.pdf) (Intel Order Number 241597, TIS Committee, Feb 1993)
- Portable Executable Format (https://bytepointer.com/resources/oleary_pe_format.htm) (Micheal J. O'Leary, Microsoft Developer Support)
- Peering Inside the PE: A Tour of the Win32 Portable Executable File Format (<https://web.archive.org/web/20080907145154/http://msdn.microsoft.com/en-us/magazine/ms809762.aspx>). Matt Pietrek, Microsoft Systems Journal, March 1994
- An In-Depth Look into the Win32 Portable Executable File Format. Matt Pietrek, MSDN Magazine. Part I, February 2002 (<https://web.archive.org/web/20021105233944/http://msdn.microsoft.com/msdnmag/issues/02/02/PE/default.aspx>); Part II, March 2002 (<https://web.archive.org/web/20021105025618/http://msdn.microsoft.com/msdnmag/issues/02/03/PE2/default.aspx>)
- The .NET File Format by Daniel Pistelli (<https://archive.today/20130130042959/http://www.ntcore.com/files/dotnetformat.htm>)
- Ero Carrera's blog describing the PE header and how to walk through (<http://blog.dkbza.org/>)
- PE Internals provides an easy way to learn the Portable Executable File Format (<http://www.andreybazhan.com/pe-internals/>)
- PE Explorer (<http://www.pe-explorer.com/>)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Portable_Executable&oldid=1348810751"